

Advanced Computer Networks — Consolidated Final Study Note

Everything for the comprehensive final in one place: the midterm recap (delays, HTTP/DNS, subnetting), the Transport Layer (Ch 3), and the Link Layer (Ch 6), each brought to life with **animated, interactive simulations**. Built from Kurose & Ross, *Top-Down Approach* (9th ed.) and the course slides.

☐ Mobile responsive

▶ Animated sims

☐ Interactive calculators

☐ Solved exam questions

☐ Print / PDF ready

PART 0 · CUMULATIVE

Midterm Recap — Delays, Application Layer, Network Layer

0.1 · Physical delays: transmission vs propagation

Total nodal delay is the sum of four pieces. Only **queueing** varies with load; the rest are fixed for a given link and packet.

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

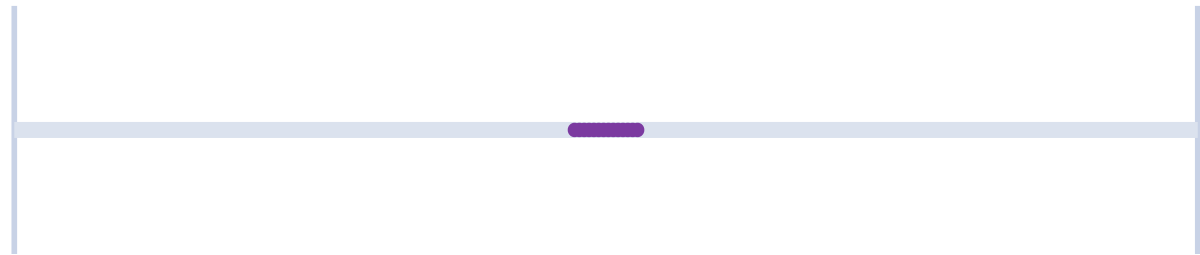
d_{proc}	: processing (check errors, pick output link)	~ constant, μs
d_{queue}	: waiting in the output buffer	VARIABLE (load)
d_{trans}	: L / R push all bits onto the link	fixed for L, R
d_{prop}	: d / s bits travel down the link	fixed for the link

The classic confusion: **transmission delay** is the time to *push* every bit of a packet onto the wire (depends on packet length L and link rate R). **Propagation delay** is the time for one bit to *travel* the distance (depends on distance d and signal speed s only). Watch it:

● Transmission vs Propagation

Sender

Receiver



pushing bits: 100%

long link: propagation dominates

traveling: 47%

$$d_{\text{trans}} = L/R$$

0.800 ms

$$d_{\text{prop}} = d/s$$

15.000 ms

total (last bit arrives)

15.800 ms

■ bits being pushed (transmission) ■ bit travelling (propagation)

Interactive simulation (open the HTML file to run): bits are pushed onto the wire over $d_{\text{trans}} = L/R$, then travel the link over $d_{\text{prop}} = d/s$.

EXAM TRAP

d_{prop} does **not** depend on L or R . A bigger packet or a faster link changes d_{trans} , never d_{prop} . Store-and-forward through one switch: $d = L/R_1 + L/R_2$ (the switch must receive the whole packet before resending).

Store-and-forward (1 switch): $d = L/R_1 + L/R_2$

Queueing (n packets ahead, x bits of current one sent): $d_{\text{queue}} = ((n+1) \cdot L - x) / R$

0.2 · Packet switching & statistical multiplexing

Packet switching **shares** a link on demand (statistical multiplexing); circuit switching **reserves** a slice per user even when idle. Sharing works because users are rarely all active at once.

● Statistical multiplexing — will it congest?



aggregate demand = 0 active $\times$ 20 = 0 Mb/s

capacity C



active now

0 / 8

$E[\text{active}] = N \cdot p$

1.2

status

OK (shared safely)

Interactive: N users each blink active p% of the time at r Mb/s on a shared C Mb/s link.

The aggregate bar turns red when demand exceeds capacity.

Circuit switching: $N_{\text{max}} = C / r$ (each user reserves r forever)

Packet switching: active users $X \sim \text{Binomial}(N, p)$, $E[X] = N \cdot p$

congestion when $X \cdot r > C \rightarrow$ usually rare

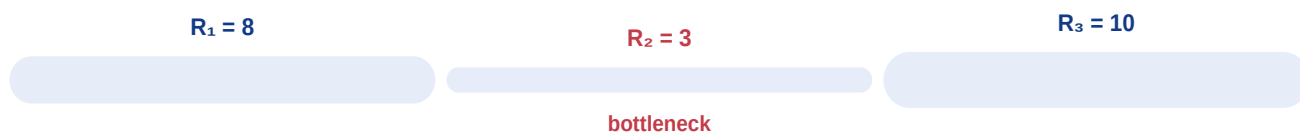
WORKED

1 Gb/s link, $r = 100$ Mb/s per active user, active 10% of time. Circuit: $N_{\text{max}} = 1000/100 = 10$. Packet with 35 users: $E[X] = 3.5$, and $P(\text{more than 10 active}) < 0.0004$ — so 35 users share safely.

0.3 · Throughput & the bottleneck link

End-to-end throughput is set by the **slowest link** on the path. A chain of pipes flows only as fast as its narrowest segment.

Bottleneck throughput = $\min(R_1, R_2, R_3)$



throughput = $\min(R_1, R_2, R_3)$

3 Mb/s

time for a 10 MB file

26.7 s

Interactive: three links of adjustable rate; the flowing bit-rate equals the minimum (the bottleneck).

throughput = $\min(R_1, R_2, \dots, R_n)$

transfer time of file $F = F / \text{throughput}$

0.4 · Application layer — HTTP, DNS, web caching

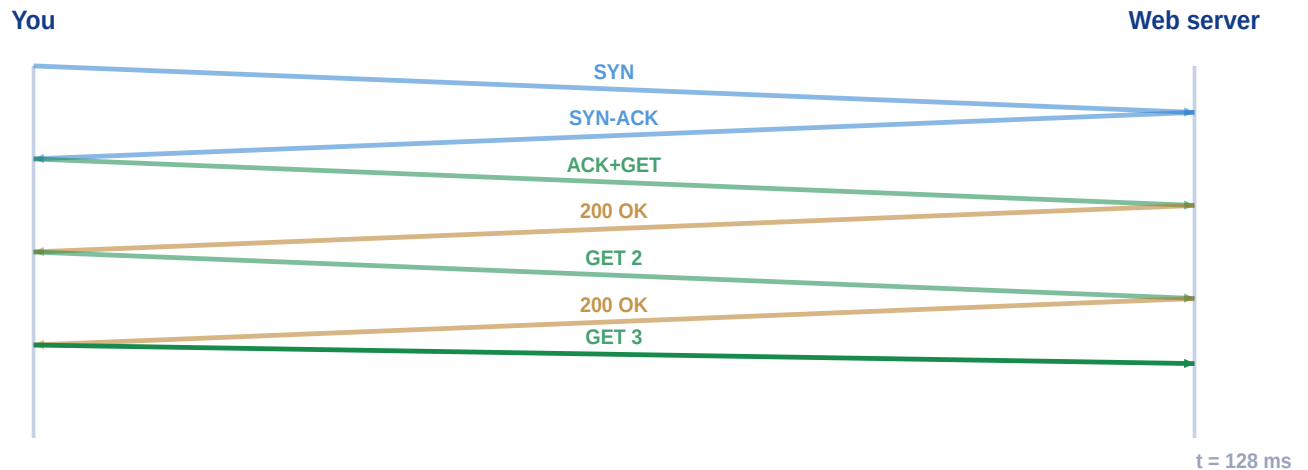
A **socket** = **IP address** : **port**. The IP finds the host (building); the 16-bit port finds the process (apartment). Applications pick TCP or UDP:

	TCP	UDP
Connection	3-way handshake	connectionless
Reliability	reliable, in-order, retransmits	best-effort, may lose/reorder
Flow / congestion control	yes / yes	no / no
Header	20+ bytes	8 bytes
Typical apps	HTTP, HTTPS, SMTP, FTP, SSH	DNS, VoIP, live video, games

HTTP: persistent vs non-persistent

HTTP runs over TCP and is **stateless** (cookies add state on top). Non-persistent opens a fresh TCP connection per object ($\sim 2 \cdot \text{RTT}$ each); persistent reuses one connection.

● HTTP page load — persistent vs non-persistent (RTT ladder)



round trips 4 RTT

total load time 160 ms

vs the other mode non-persistent $\approx 6 \text{ RTT}$

■ TCP handshake ■ HTTP request ■ response

Interactive time-sequence ladder: toggle persistent/non-persistent and object count to see how round trips add up. Non-persistent repeats the TCP handshake per object.

Web caching — access-link math

Offered traffic $T = \lambda \cdot L$ Utilisation $U = T / C$ (as $U \rightarrow 1$, delay $\rightarrow \infty$)
 With cache hit rate h , only misses use the link: $T' = (1-h) \cdot \lambda \cdot L$
 Avg delay = $(1-h) \cdot d_{\text{miss}} + h \cdot d_{\text{hit}}$

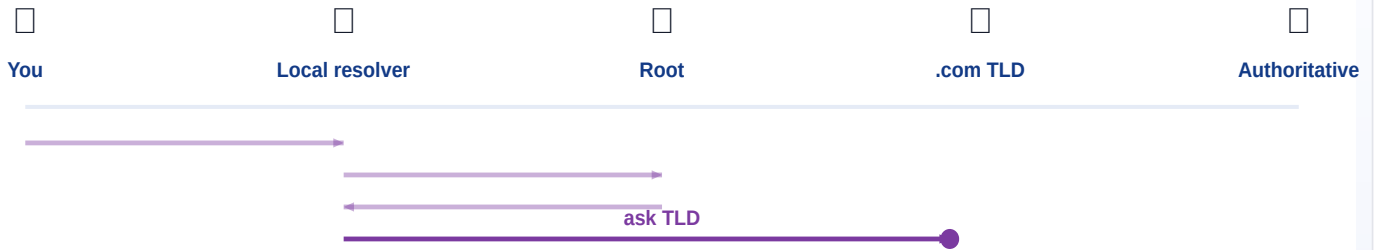
WORKED

Access link 1.54 Mb/s, object 100 kb, 15 req/s, RTT 2 s. $T = 1.5 \text{ Mb/s} \rightarrow U = 0.97$ (minutes of delay). Cache $h = 0.4 \rightarrow T' = 0.9 \text{ Mb/s} \rightarrow U' = 0.58$, avg delay $\approx 1.2 \text{ s}$.

DNS — the Internet's phone book

Translates names $\rightarrow$ IPs. It is **distributed & hierarchical** (root $\rightarrow$ TLD $\rightarrow$ authoritative) to avoid a single point of failure, huge load, distant lookups, and unmaintainability. Records: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), NS (name server).

● DNS resolution walk (iterative)



mode

cache miss — hierarchical walk

step

ask TLD

Interactive: watch a query walk local resolver → root → TLD → authoritative and return the IP; a cache hit short-circuits it.

0.5 · Network layer — IP addressing, subnetting, VLSM, NAT

IPv4 is **32 bits** = four octets (0–255), split into a network part (left) and host part (right) by the mask / CIDR . Try the calculator — it does the block-size trick and shows the binary split:

Subnet calculator (block-size method)

IP address

172.16.35.123

Prefix /m

20

Mask

255.255.240.0 (/20)

Network

172.16.32.0

Broadcast

172.16.47.255

Host range

172.16.32.1 – 172.16.47.254

Usable hosts ($2^n - 2$)

4,094

Block size

16

10101100.00010000.00100011.01111011

Network bits shown in blue, host bits in amber. Uses the block-size method.

Interactive: enter an IP and prefix; get network, broadcast, host range, usable hosts, and a binary network/host split.

host bits $n = 32 - m$ usable hosts = $2^n - 2$ (minus network + broadcast)
block size = $256 - (\text{last non-255 mask octet})$ network = block start
broadcast = next block start - 1

Classful (identify by first octet): A 0–127 (/8), B 128–191 (/16), C 192–223 (/24), D 224–239 (multicast), E 240–255 (reserved). Private: 10/8, 172.16/12, 192.168/16.

VLSM — variable length subnet masks

Give each subnet its own mask, just big enough. **Recipe:** sort requirements largest → smallest; pick the smallest n with $2^n - 2 \geq \text{hosts}$; assign the block; start the next subnet right after the previous broadcast.

LAN	Need	$2^n - 2$	Prefix	Network	Broadcast
LAN1	100	126 ($n=7$)	/25	192.168.10.0	192.168.10.127
LAN2	50	62 ($n=6$)	/26	192.168.10.128	192.168.10.191
LAN3	20	30 ($n=5$)	/27	192.168.10.192	192.168.10.223

EXAM TRAP

Allocate the **largest subnet first**, or a small one will land on a boundary the big one needs. Always subtract 2 (network + broadcast).

NAT — network address translation

Lets a whole private network share **one** public IPv4 address. The router rewrites (source IP, source port) outbound and reverses it inbound, tracking each mapping in a NAT table. Pros: conserves IPv4, hides topology. Cons: breaks end-to-end addressing.

PART A · CHAPTER 3

The Transport Layer

A.1 · Multiplexing & demultiplexing

The transport layer extends host-to-host (IP) delivery to **process-to-process**. The receiver *demultiplexes* each segment to the right socket using header fields.

- **UDP demux = 2-tuple** (dest IP, dest port). Same dest port from different sources → same socket.
- **TCP demux = 4-tuple** (src IP, src port, dst IP, dst port). Each client gets its own connection socket even on port 80.

● Demultiplexing — UDP (2-tuple) vs TCP (4-tuple)

Server (port 80)

src A:5000

socket (A,·,B,80)

socket (C,·,B,80)

demux key

(src IP, src port, dst IP, dst port) — 4-tuple

result

A and C get SEPARATE sockets

Interactive: segments arrive and are routed to sockets. Toggle UDP/TCP to see 2-tuple vs 4-tuple demultiplexing.

EXAM TRAP

For UDP the source is just a return address and does not change which socket receives data. For TCP all four fields matter.

A.2 · Reliable data transfer & pipelining

We build reliability on an unreliable channel step by step. **Sequence numbers** catch duplicates; **timers** catch loss. rdt3.0 is *stop-and-wait* — one packet in flight — which wastes the link.

Stop-and-wait utilisation: $U = (L/R) / (RTT + L/R)$

Example: 1 Gb/s, RTT 30 ms, L 8000 b $\rightarrow L/R = 8 \mu s \rightarrow U \approx 0.027\%$ (terrible!)

● Stop-and-wait vs Pipelining — link utilisation

Stop-and-wait (N=1)



Pipelined (N=4)



stop-and-wait util (RTT/(L/R)=30)

3.2%

pipelined util (N=4)

12.9%

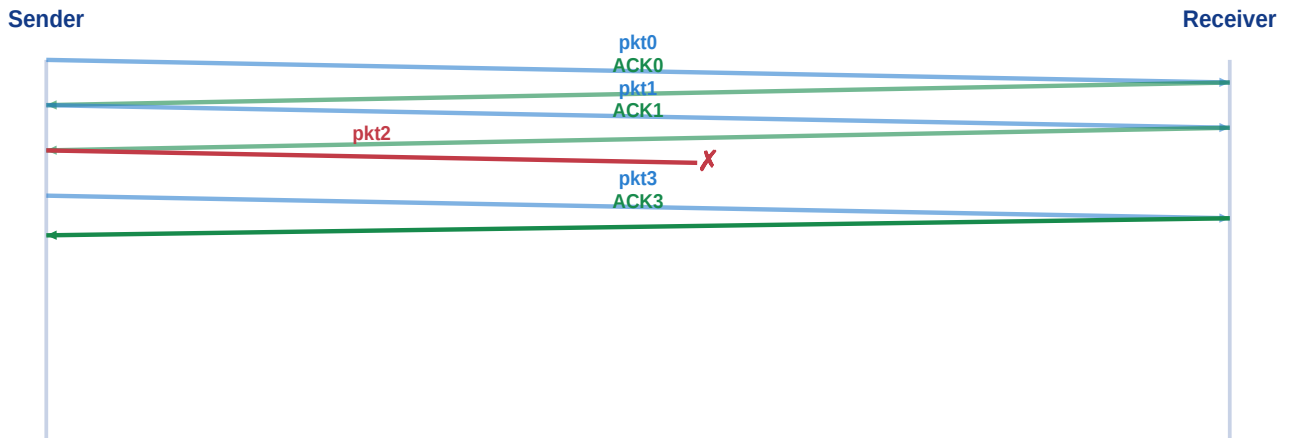
■ data ■ ACK

Interactive: compare one-in-flight stop-and-wait to a pipelined sender (window N).
Utilisation bar fills much more with pipelining.

Go-Back-N vs Selective Repeat

	Go-Back-N	Selective Repeat
ACKs	cumulative	individual
Timers	one (oldest unACKed)	one per unACKed packet
On loss	resend packet n and all higher	resend only the lost packet
Receiver buffer	none (discard out-of-order)	buffers out-of-order
Window rule	$N \leq 2^k - 1$	$N \leq 2^{k-1}$ (half the seq space)

Go-Back-N vs Selective Repeat — drop a packet and watch



on loss of pkt 2

re-send pkt 2,3,4 (all higher)

receiver

discards out-of-order

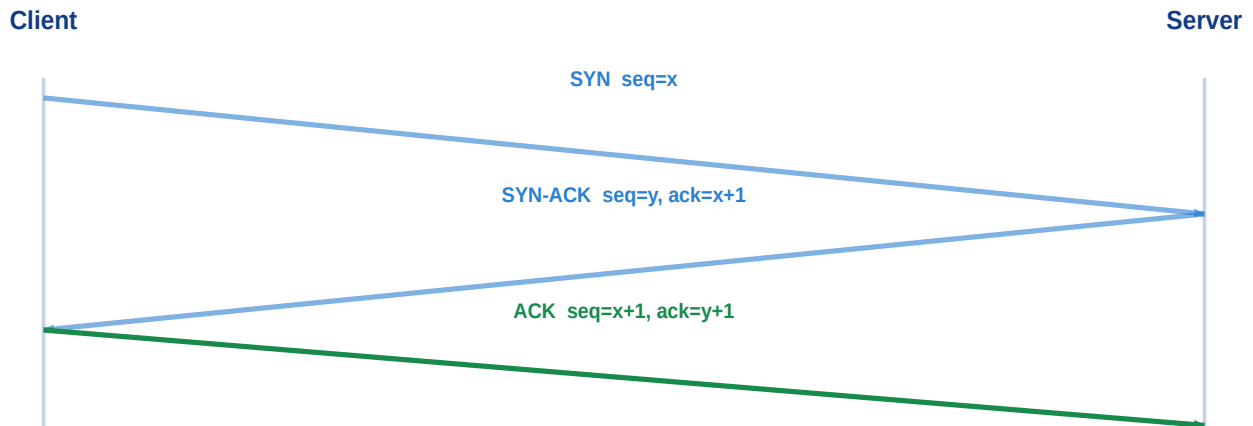
■ sent OK ■ lost ■ buffered (SR) ■ ACK

Interactive: toggle GBN/SR, then drop a packet. GBN re-sends everything from the loss; SR re-sends only the missing one.

A.3 · TCP — connection, sequence numbers, flow control

TCP is a reliable, in-order **byte stream**: point-to-point, full-duplex, pipelined, with cumulative ACKs. Seq # = byte-stream number of the first data byte; ACK # = next byte expected.

TCP 3-way handshake (and why 2-way fails)



why 3
messages?

Both sides confirm each other's initial seq #; a 2-way handshake can leave half-open / duplicate connections.

Interactive: play the SYN → SYN-ACK → ACK exchange with sequence numbers; both sides reach ESTABLISHED.

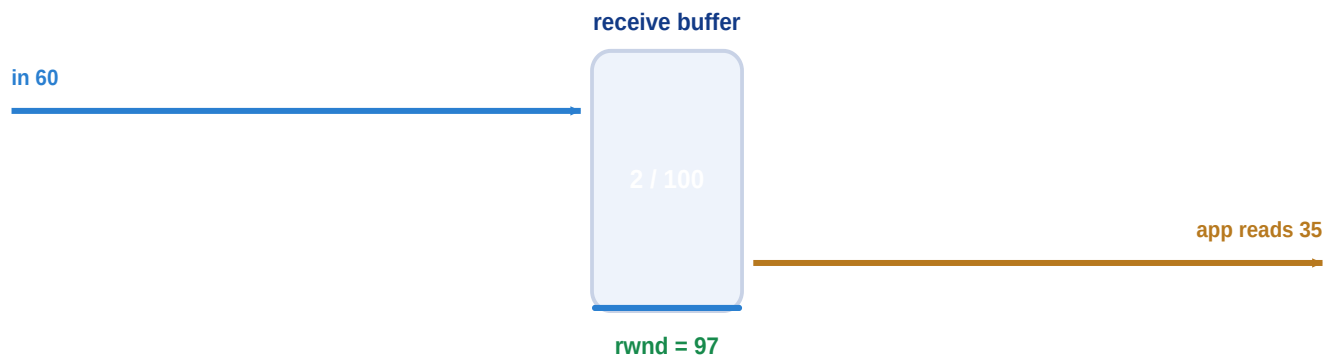
```
EstimatedRTT = (1-α)·EstimatedRTT + α·SampleRTT      (α = 0.125)
DevRTT        = (1-β)·DevRTT + β·|SampleRTT - EstimatedRTT|  (β = 0.25)
TimeoutInterval = EstimatedRTT + 4·DevRTT
```

Fast retransmit: 3 duplicate ACKs → resend the missing segment before the timer fires.

Flow control (protects the receiver)

The receiver advertises free buffer space in **rwnd**; the sender caps in-flight data to rwnd so the receive buffer never overflows.

● Flow control — rwnd tracks the receive buffer



advertised rwnd	97
sender capped to	60 ($\leq$ rwnd)
buffer overflow?	never (flow control)

Interactive: sender rate vs app read rate fill/drain the receive buffer; rwnd (advertised free space) throttles the sender.

EXAM TRAP

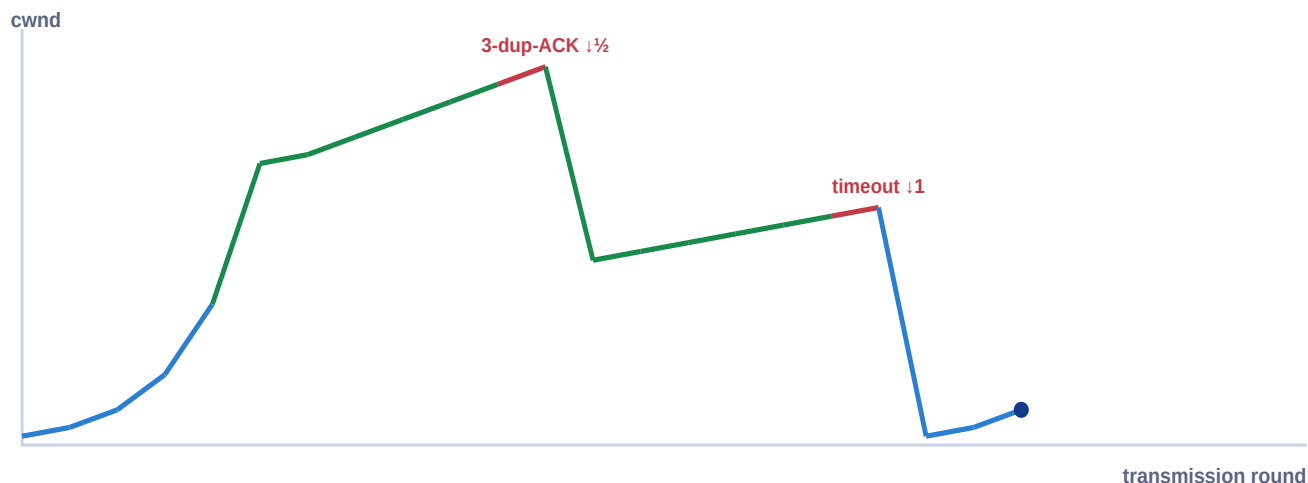
Flow control (rwnd) protects the **receiver**. Congestion control (cwnd) protects the **network**. Different mechanisms.

A.4 · Congestion control — slow start & AIMD

TCP probes for bandwidth with a congestion window **cwnd**; it sends `min(cwnd, rwnd)`.

Slow start ramps exponentially to **ssthresh**, then congestion avoidance adds linearly (AIMD). On loss, `ssthresh = cwnd/2`.

● Congestion window — slow start, AIMD sawtooth, loss events



round 21

cwnd 4 MSS

phase slow start (exponential)

■ slow start (exp) ■ congestion avoidance (linear) ■ loss

Interactive/animated cwnd-vs-round graph: exponential slow start, linear congestion avoidance, halving on triple-dup-ACK, reset to 1 on timeout.

READ A P40 GRAPH

Doubling per round = **slow start**. +1 per round = **congestion avoidance**. Window **halved** on loss = triple-duplicate-ACK (Reno). Window **dropped to 1** = timeout. On any loss, ssthresh = cwnd/2.

AIMD: additive increase +1 MSS/RTT ; multiplicative decrease $\times \frac{1}{2}$ on 3-dup-ACK
avg TCP throughput $\approx (3/4) \cdot W / \text{RTT}$ fairness: K flows $\rightarrow \sim R/K$ each

PART B · CHAPTER 6

The Link Layer & LANs

B.1 · Error detection & correction

The sender adds redundant EDC bits. **2-D parity** can not only detect but *correct* a single-bit error by pinpointing its row and column. Flip a bit and watch it get located:

Two-dimensional parity — detect & correct a flipped bit

0	1	1	0	0
0	1	1	0	0
0	0	0	1	1
0	1	1	1	1
0	1	1	0	

No errors yet — tap a bit to inject one.

Tap any blue data bit to flip it (inject an error). The mismatched row + column pinpoint it.

Interactive: click any data bit to flip it; the mismatched row and column parity pinpoint the error for correction.

CRC is stronger: pick r check bits R so that $\langle D, R \rangle$ is divisible by generator G (mod-2). The receiver divides by G ; a nonzero remainder means error. Detects all bursts $\leq r$ bits (used by Ethernet, WiFi).

CRC — mod-2 polynomial division

Data D

101110

Generator G

1001

```
1011 XOR 1001 = 0010
0101 (leading 0, bring down)
1010 XOR 1001 = 0011
0110 (leading 0, bring down)
1100 XOR 1001 = 0101
1010 XOR 1001 = 0011
```

CRC remainder $R = 011$. Transmit $\langle D, R \rangle = 101110011$. Receiver divides by G ; remainder 0 $\Rightarrow$ no error detected.

Interactive: enter data D and generator G; step through the XOR long-division to get the CRC remainder R.

B.2 · Multiple access — CSMA/CD

On a shared broadcast medium, two simultaneous transmissions **collide**. CSMA = listen before transmit; CSMA/CD also aborts on collision detection and backs off. Collisions still happen because of propagation delay (a node may not yet hear the other).

● CSMA/CD — collision, jam, binary exponential backoff



phase

collision + jam

after m-th collision

$K \in \{0 \dots 2^m - 1\}$, wait $K \cdot 512$ bit-times

■ node A signal ■ node C signal ■ collision ■ backoff

Interactive: two nodes sense and transmit; overlapping signals collide, a jam is sent, and each backs off a random $K \cdot 512$ bit-times.

Slotted ALOHA max efficiency = $1/e \approx 37\%$ Pure ALOHA = $1/(2e) \approx 18\%$
CSMA/CD backoff: after m-th collision pick $K \in \{0 \dots 2^m - 1\}$, wait $K \cdot 512$ bit-times
Collision if two start within d_{prop} : yes when $d_{\text{prop}} < L/R$

B.3 · MAC addressing & ARP

A 48-bit **MAC address** (flat, in NIC ROM) moves a frame between interfaces on the same subnet. **ARP** maps a known IP → its MAC: broadcast the query, get a unicast reply.

● ARP — broadcast query, unicast reply

shared LAN

ARP query broadcast: FF:FF:FF:FF:FF:FF "who has D's IP?"



A



B



C



D (target)

query

broadcast (unknown MAC)

reply

unicast (MAC now known)

■ broadcast query ■ unicast reply

Interactive: A broadcasts "who has IP?" to all nodes; only the owner replies unicast, and A caches the mapping.

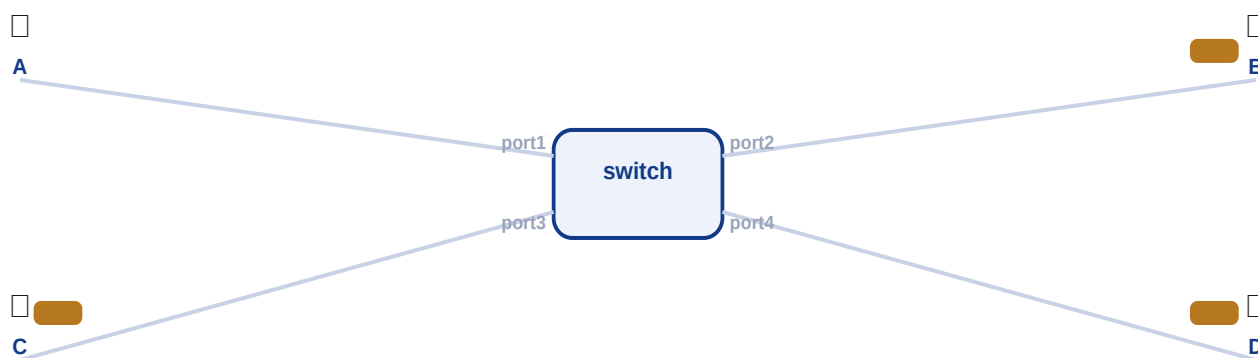
CORE IDEA

Across the Internet, **IP addresses stay fixed end-to-end** while **MAC addresses are rewritten at every hop**. ARP query = broadcast; ARP reply = unicast.

B.4 · Ethernet switches — self-learning

A switch is a transparent, plug-and-play, store-and-forward layer-2 device. It **learns** which MAC lives on which port from the source address of arriving frames, then forwards (or floods if the destination is unknown).

● Self-learning switch — table fills, floods then forwards



unknown dest → FLOOD all ports

switch table

(empty)

this frame A → C

flood (dest unknown)

Interactive: send frames between hosts; the switch table learns source ports, floods unknown destinations, and forwards known ones on a single port.

Switch	Router
Layer 2, examines MAC	Layer 3, examines IP
Table by flooding + learning	Table by routing algorithms
Does not change TTL	Decrements TTL each hop

VLANs split one physical switch into isolated broadcast domains; 802.1Q adds a 4-byte tag with a 12-bit VLAN ID → up to $2^{12} = 4096$ VLANs.

B.5 · A day in the life of a web request

The grand synthesis, laptop → www.google.com:

1. **DHCP** (broadcast, UDP/IP/Ethernet) → laptop gets its IP, gateway IP, DNS server IP.
2. **ARP** → laptop learns the first-hop router's MAC.
3. **DNS** query (UDP) → returns google's IP.
4. **TCP** 3-way handshake to the web server.
5. **HTTP** GET over TCP → server returns the page → browser renders.

COMPANION

For the full hop-by-hop packet journey (every router rewriting MACs, TTL counting down, IP constant), open `browsing_the_internet.html` in this folder.

PART C · ASSIGNED

Solved Exam Questions

Chapter 3 — Review questions

R3. How is a UDP socket fully identified? A TCP socket? Difference?

UDP: a 2-tuple (dest IP, dest port). **TCP:** a 4-tuple (src IP, src port, dest IP, dest port). UDP ignores the source for demux; TCP uses all four, so different clients on port 80 get different sockets.

R4. Why might a developer choose UDP over TCP?

No congestion throttling (rate-sensitive media), no handshake RTT, no per-client connection state (more clients), smaller header — and the app can add exactly the reliability it needs (e.g. HTTP/3 over UDP).

R14. True or False?

#	Statement	Answer	Why
a	B won't ACK A because it can't piggyback	False	TCP sends pure ACK segments anytime.
b	rwnd never changes	False	It changes as the buffer fills/drains.
c	Unacked bytes $\leq$ receive-buffer size	True	Flow control caps in-flight $\leq$ rwnd.
d	Next seq # is necessarily m+1	False	Next = m + bytes of data.
e	Header has an rwnd field	True	The 16-bit receive-window field.
f	SampleRTT 1 s $\Rightarrow$ Timeout $\geq$ 1 s	False	Timeout uses smoothed EstRTT + 4·DevRTT.
g	seq 38 + 4 bytes $\Rightarrow$ ACK is 42	False	42 is what B sends back, not A's own ACK.

R15. Two segments: seq 90 then seq 110.

(a) Data in first = $110 - 90 = 20$ bytes. (b) First lost, second arrives → cumulative ACK still requests byte **90** (a duplicate ACK).

R18. On timeout, ssthresh = $\frac{1}{2}$ its previous value — T/F?

False. ssthresh = $\frac{1}{2}$ of the current *cwnd*, not of the previous ssthresh. On a timeout, *cwnd* then resets to 1 MSS and slow start restarts.

Chapter 3 — Problems

P1. Two Telnet clients A, B → Server S (port 23).

Segment	Source port	Dest port
a. A → S	467 (any ephemeral)	23
b. B → S	513 (any ephemeral)	23
c. S → A	23	467
d. S → B	23	513

(e) Different hosts → source ports **may match** (IPs differ, so the 4-tuples differ). (f) Same host → source ports **must differ**.

P27. B has received through byte 126; A sends 80 then 40 bytes, first seq = 127.

- (a) First segment: seq **127**, src 302, dst 80.
- (b) Second: seq **207** ($=127+80$), 40 bytes.
- (c) First arrives first → ACK **207** (src 80, dst 302).
- (d) Second arrives first (gap) → ACK still **127**.

```
First lost, second arrives, then retransmit:
A --seq=127,80B--X (lost)
A --seq=207,40B--> arrives out of order
    <--ACK=127 (gap ACK)
(timeout / 3 dup-ACKs)
A --seq=127,80B--> gap filled
    <--ACK=247 (127+80+40 - cumulative)
```

P28. GBN cumulative-ACK robustness.

TCP acknowledges the next expected in-order byte, so one ACK can cover several earlier segments. If ACK1/ACK2 are lost but ACK3 (higher, cumulative) arrives, the sender advances past all three — no retransmission. Retransmit only on timeout or 3 dup-ACKs.

P40. cwnd-vs-round graph (standard: rounds 1–26, ssthresh 32 then 21).

- (a) Slow start: rounds [1,6] and [23,26] (exponential).
- (b) Congestion avoidance: [6,16] and [17,22] (linear).
- (c) After round 16: halved, stays >1 → **triple-dup-ACK**.
- (d) After round 22: drops to 1 → **timeout**.
- (e) Initial ssthresh = **32** (slow start ends at cwnd 32).
- (f) After round 18: ssthresh = $42/2 = 21$.
- (h) Loss after round 26 by 3-dup-ACK → cwnd and ssthresh both = **W/2**.

Chapter 6 — Review questions

R1. Transportation analogy: what is the frame?

The **transportation mode** (car, bus, train, plane) carrying the passenger (datagram) over one leg — as a datagram rides inside a frame across one link.

R2. If every link were reliable, is TCP reliability redundant?

No. Per-link reliability only covers one link. Datagrams can still arrive *out of order* (different routes) or be *lost* (routing loops, buffer overflow, equipment failure). TCP still provides the ordered, loss-free end-to-end byte stream.

R3. Link-layer services; which also in IP? TCP?

Service	In IP?	In TCP?
Framing	✓ (encapsulation)	✓
Link access	—	—
Reliable delivery	✗	✓
Flow control	✗	✓
Error detection	✓ (header checksum)	✓
Error correction	✗	✗
Full duplex	—	✓

R4. Two nodes transmit length-L at rate R; collision if $d_{\text{prop}} < L/R$?

Yes. If $d_{\text{prop}} < L/R$ the sender is still transmitting when the other's signal arrives, so the signals overlap on the medium → collision.

R10. A → B unicast on a broadcast LAN; does C process/pass up? Broadcast?

Unicast to B's MAC: C's adapter processes then **discards** (dest MAC ≠ its own) — not passed up. Broadcast MAC: C **processes and passes up** to its network layer.

R11. Why ARP query broadcast but reply unicast?

The querier doesn't know which MAC owns the IP, so it must ask everyone (broadcast). The responder already learned the querier's MAC from the query, so it replies directly (unicast) without bothering the whole LAN.

REFERENCE

Formula Sheet & Exam Traps

Transmission delay	$d_{trans} = L / R$
Propagation delay	$d_{prop} = d / s$
Store-and-forward	$d = L/R_1 + L/R_2$
Queueing (n ahead)	$d_{queue} = ((n+1) \cdot L - x) / R$
Throughput	$\min(R_1, \dots, R_n); \text{ file time} = F / \text{throughput}$
Stop-and-wait util	$U = (L/R) / (RTT + L/R)$
RTT estimate	$EstRTT = (1-\alpha)EstRTT + \alpha \cdot SampleRTT \quad (\alpha=0.125)$
RTT deviation	$DevRTT = (1-\beta)DevRTT + \beta SampleRTT - EstRTT \quad (\beta=0.25)$
Timeout	$EstRTT + 4 \cdot DevRTT$
TCP throughput	$\approx (3/4) \cdot W / RTT$
SR window	$N \leq (\text{seq space})/2$
Slotted / Pure ALOHA	$1/e \approx 0.37 \quad / \quad 1/(2e) \approx 0.18$
CSMA/CD backoff	$K \in \{0 \dots 2^m - 1\}; \text{ wait } K \cdot 512 \text{ bit-times}$
Subnet hosts	$2^n - 2; \text{ block} = 256 - \text{last mask octet}$
VLANs (802.1Q)	$2^{12} = 4096$
MAC / IPv4 / IPv6	$2^{48} / 2^{32} / 2^{128} \text{ addresses}$

Top exam traps

1. Flow control (rwnd) protects the receiver; congestion control (cwnd) protects the network.
2. Next TCP seq # = current seq # + bytes of data, **not** +1.
3. On loss ssthresh = cwnd/2. Timeout → cwnd = 1 MSS. 3-dup-ACK → cwnd = cwnd/2 (Reno).
4. UDP demux = 2-tuple; TCP demux = 4-tuple.
5. IP addresses stay fixed end-to-end; MAC addresses change every hop.
6. ARP query = broadcast; ARP reply = unicast.
7. Switch: unknown dest → flood; known dest same port → drop; else forward on one port.
8. Slow start = exponential; congestion avoidance = linear.
9. d_prop depends on distance only, never on L or R. Only d_queue is variable.
10. Throughput = min of link rates (bottleneck).
11. Usable hosts = $2^n - 2$; block size = 256 – last mask octet.
12. VLSM: allocate largest subnet first. Non-persistent HTTP $\approx 2 \cdot RTT$ per object.

